

Documentação Técnica e Lógica do E-commerce

Relatório de Revisão das Implementações Recentes

Este documento detalha, de forma isolada e didática, toda a lógica de arquitetura e código que foi desenvolvida nas últimas atualizações do sistema. O objetivo é fornecer clareza e segurança sobre como cada engrenagem da plataforma está operando.

1. Correção de Integridade: Pedidos e Endereços (Prisma)

O Problema Original

O Prisma (nosso ORM que gerencia o Banco de Dados) estava acusando um erro no momento de buscar a lista de pedidos (`prisma.order.findMany()`). O código da página tentava incluir os dados do endereço (`include: { address: true }`), porém a tabela de `Order` não possuía o "vínculo oficial" com a tabela `Address` detalhado no Schema.

A Solução Lógica

Fomos ao arquivo central do banco de dados (`prisma/schema.prisma`) e declaramos explicitamente a relação.

- **Passo 1:** Adicionamos o campo de relacionamento `address` na tabela `Order`, ligando a chave estrangeira `addressId` ao `id` da tabela `Address`.
- **Passo 2:** Para garantir a reciprocidade que a ferramenta exige, informamos à tabela `Address` que ela pode possuir um array de `orders` `Order[]`.
- **Passo 3:** Rodamos a recriação das tipagens do ambiente (`npx prisma generate` e `db push`), fazendo com que o TypeScript e o Banco entendessem essa conversa. Agora o painel pode puxar o endereço de entrega completo de um pedido em uma única consulta pesada, reduzindo o processamento do servidor.

2. Padrão CRUD Operacional: Módulo de Categorias

A entidade de Categorias da loja não contava com um sistema completo de gestão. O administrador precisava do controle total (Criar, Ler, Atualizar, Deletar - CRUD).

Arquitetura de Ações (Backend)

No arquivo `category-actions.ts`, implementamos o lado servidor da lógica:

- `updateCategory` : Busca uma categoria pelo `id` exclusivo, recebe o envio de um formulário (`FormData`) e atualiza as colunas de `nome` e `descrição`. Por trás dos panos, o sistema refaz automaticamente a `slug` (A URL amigável, ex: "Moda Verão" se torna "moda-verao").
- `deleteCategory` : **Lógica de Segurança Crítica.** Antes de confirmar um `delete()`, o código realiza um `count` (contagem) para verificar quantos produtos estão atrelados ao nome desta categoria. Se houver 1 ou mais, o Backend aborta a exclusão e joga um erro formatado para o usuário. Isso previne que a loja fique com itens "órfãos" sem categoria, o que quebraria a vitrine.

Interface e Reatividade (Frontend)

Foi criado o componente visual `CategoryActionsClient.tsx`. Ele funciona puramente do lado do cliente (`"use client"`) para lidar com animações de Modais Pop-ups e estado de Carregamento (Loading spinners), separando-se

do carregamento bruto da página server-side. Ele injeta os dois ícones (Editar e Lixeira) e se conecta com os Toast notifications (Avisos de sucesso verde/vermelho no canto da tela).

3. Gestão Logística Avançada: Estoque por Lote (FIFO)

Para elevar a plataforma ao padrão de um ERP corporativo, o estoque migrou de uma configuração "solta" (apenas alterando um número de +5 para -5) para um controle rastreável. Em suma, o produto principal agora consolida o estoque, mas a entrada oficial de mercadorias é dada pelos lotes!

Entidade Nova: `InventoryBatch` (Lote de Inventário)

Criamos a tabela no banco projetada para receber:

1. `batchCode` : Código da nota/lote da transportadora.
2. `initialQuantity` e `availableStock` : Isso permite que não mudemos o registro de quantas unidades vieram na Van. Se vieram 100 itens (`initialQuantity`), mas só restam 30, o `availableStock` ficará como 30. O histórico permanece imaculado.
3. Informações Financeiras: Custo Unitário e Nome do Fornecedor vinculado a essa rodada de compras.

O Algoritmo FIFO (First-In, First-Out | O Primeiro que entra é o Primeiro que sai)

Construímos a inteligência `consumeStockFIFO` em `inventory-actions.ts` que entra em ação no momento em que uma compra online é concluída na plataforma.

- **A Lógica:** Quando um usuário compra 3 Camisetas, o serviço não deduz apenas "3 unidades do Produto". O sistema de banco de dados busca uma lista de `InventoryBatch` relacionados a essa mercadoria, arranjados pela "Data de Criação Mais Antiga" ascendente (`orderBy: createdAt: 'asc'`).
- Se o Lote "A" (criado em Janeiro) possui 2 peças em `availableStock` , o sistema esgota o Lote "A" deixando-o com 0 peças.
- Falta 1 peça para inteirar as 3 camisas da compra. O código segue em Loop (laço de repetição) e entra no Lote B (criado em Fevereiro), deduzindo 1 peça dele.
- Todos os dados de Total do produto (`Product.stock`) viram apenas uma "Soma" (`_sum`) de todos os estoques atrelados a esses lotes para exatidão impecável e zero dessincronização em alta carga.

Dashboard Reestruturada

Na página central `/admin/inventory` , aplicamos o componente `<FadeIn>` revelando não só produtos em falta, como a tabela Histórica. Ela renderiza a lista limpa das últimas mercadorias bipadas contendo preço da compra, nome e quantos já foram gastos (`Available / Initial`). O botão principal que alimenta isso agora é o `RestockButton.tsx` .

Documento gerado e revisado para os padrões de Desenvolvimento Local de Produção.